

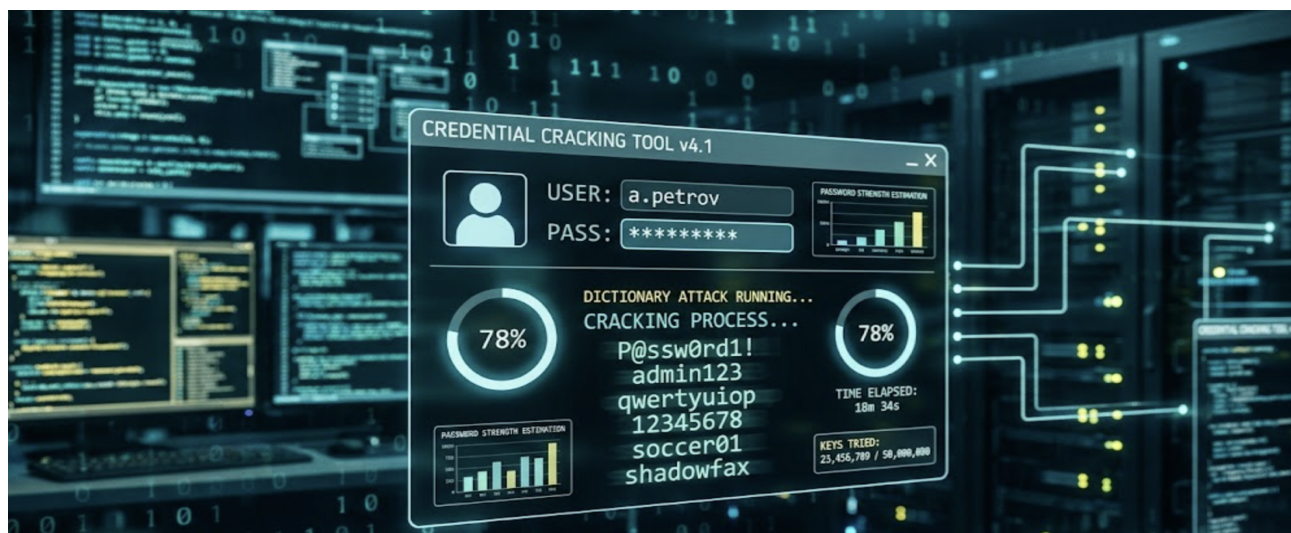
Computer and Network Security

Lab 04 (“Cracking Credentials”)

Disclaimer. Hacking without permission is illegal. These labs are strictly educational for learning about cyber-security in the areas of ethical hacking and penetration testing so that we can protect ourselves against the real hackers. Hence, any attack technique, tools, and technology, learned during these labs should only be launched on systems you either own or have permission to launch attacks on. Don't wrong your classmates by launching attacks against them—they will know in the judgement day and you will pay for that. Please be professional, wise, and mature.

Lab Note. In Lectures 4, you have learned that the use of passwords to protect accounts is one of the most important and easy to implement “what-you-know”-type of authentication mechanisms. Also, throughout the lecture and multiple tutorial sessions, you have learned that password-based authentication is subject to various forms of attacks.

Description. In this lab, you will learn how to use password cracking software to perform online and offline cracking of credentials to access services and protected files.



Task 1 (Cracking Credentials with Hydra) — Online Cracking

In this part, you are going to use **hydra**, a command-line based login cracker that support various protocols. You will try to crack the credentials that are used to connect to various services, including, ssh, telnet, and http. Note that a GUI-based version exists (xHydra).

1. **SSH.** As you know, SSH is an application-layer protocol (in the TCP/IP suite) that allows a user to connect to a remote computer and run commands on that computer. To that end, the remote computer must be running an SSH server (e.g., Openssh-server on GNU/Linux), whereas the user should use an SSH client tool (e.g., Openssh-client).

- On your client machine, use **nmap** to make sure that the target computer (SSH server) is indeed running ssh (e.g., `$nmap -sV -p 22 IP`, where IP is the IPv4 address of your ssh server). If you find the port to be open, then the ssh server is up and running. Use **hydra** along with the two provided files: **users.txt** and **passwords.txt** as follows (make sure that **users.txt** contains the username of the ssh client, for which you want to crack the password, and that **passwords.txt** contains their password too):

```
$hydra IP ssh -Vv -L ./users.txt -P ./passwords.txt -e s  
-t 20
```

- The **hydra** tool will use all combinations of usernames from **user.txt** and passwords from **passwords.txt**, to establish ssh connections with the server. If a connection is successful, **hydra** will let you know the credentials.
- How much time it took for **hydra** to crack the credentials? Could you try the same attack with parameter **-t 50**? What about this time?
- You can use **hydra** along with the third file: **credentials.txt** as follows:

```
$hydra IP ssh -Vv -C credentials.txt -e s -t 20
```

- How do you classify the credential-cracking attack you just performed?
- Run the attack again but use **Wireshark** to observe and analyze the packets that were exchanged during the attack.

- Use the credentials you cracked above (e.g., omar:ensia2024), and use Wireshark to analyze the different packets exchanged during the attack when you run the following command:

```
$hydra IP ssh -Vv -l omar -p ensia2024 -e s -t 1
```

- Use Internet and look for the message sequence chart for ECDH in SSH. Then, try to map the standard messages of the protocol with the one you capture in **Wireshark** when you ran the previous commands (Ignore technical details). And, don't worry the details of the protocol will be discussed in the lecture.

2. **Telnet.** In this part, you will try to use **hydra** to crack the credentials of a telnet user in the same way you did with ssh above.

- Use **hydra** along with the credentials file **credentials.txt** to crack the credentials of the registered user:

```
$hydra IP telnet -Vv -C credentials.txt -e s -t 20
```

- If **hydra** managed to crack the credentials, then congrats. Otherwise, if it failed, could you use **Wireshark** and follow up the messages to determine what went wrong?

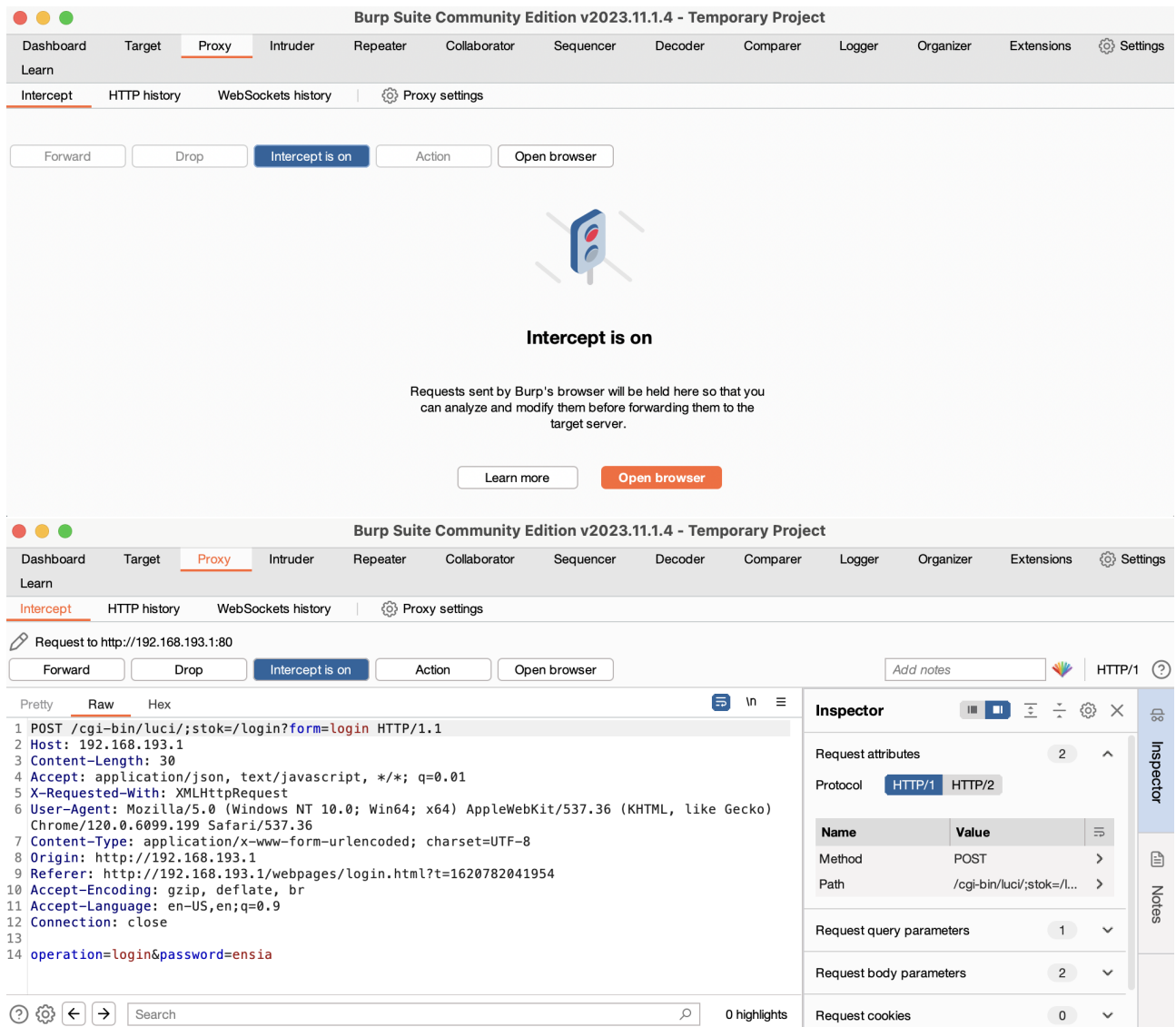
3. **HTTP.** In this part, we will use **hydra** to crack the credentials of a web service (e.g., a web-application). First, we need to obtain the exact HTTP request that is sent to the server when trying to login. To that end, we are going to use **Burp suite**. Burp Suite is a set of security testing tools widely used for web application security testing. It provides a range of features for security professionals, penetration testers, and developers to identify, analyze, and address security vulnerabilities in web applications.

Open **Burp**, click on next, then launch burp. After that, go to the “Proxy” tab and make sure that the “Intercept” function is on (viz., Figure 1 below).

You can configure your browser to use Burp as a forward proxy so that it intercepts your HTTP requests. Different configurations exist for each web-browser. E.g., for Firefox, the configuration happens at the browser level, whereas on Google Chrome's browser, it happens at the operating system's level. Yet, you

can use the Burp browser by clicking on “Open browser” on the Burp’s proxy tab (viz., Figure 1 below).

You can type in the IP address (or URL) of the web-server on the Burp’s browser and you will see your HTTP request being intercepted by the proxy. The web-page will not get fully loaded into you hit the “Forward” button several times. You could then type incorrect credentials (just use some random credentials) and hit forward until you intercept the HTTP POST request that carries the login credentials (viz., Figure 2 below).



The other three things you should need are: (1) the URL of the login’s web-page (e.g., /webpages/login.html), (2) the request body, that you can get from the intercepted request (e.g., username=admin&password=ensia2024 or operation=login&password=ensia), and (3) a message that allows you to identify a failed login, e.g., “Incorrect password”.

- First, you are going to crack the password of the router **AI388 Lab (4G)**'s web-server. Make sure to be connected to the Wi-Fi network **AI388 Lab (4G)**.
- Then, use **burp**, in proxy interceptor mode, to capture any HTTP request. Use the **burp**'s web-browser to navigate to 192.168.2.1 (the router's IPv4 address) for the login-page. The **burp**'s proxy will intercept your request. Keep clicking on "Forward" till the output is blank. Then, input some incorrect credentials and submit them. You would see the HTTP request carrying the submitted credentials. For this router, it is using the GET method to submit the input of forms. You are looking for the body request string, which has the a string-pattern that looks to the following:

```
/log/in?un=admin&pw=ensia&rd=...&rd2=...&Nrd=...&Nlmb=
HTTP/1.1
```

- For this router, the username is fixed to **admin**, which is not that secure, as it simplifies the task for the attacker. They only need to find the password. Use **hydra** to crack the password. Below is an example that was run on the same router (however, check the request body, it may be different). Also, there is a column (:) between the path of the login page, i.e., **/log/in**, and the parameters. The password value is set to **^PASS^** to tell **hydra** to use the passwords from the provided list and try them on this variable parameter. Finally, after the parameters, you should put another colon, and write the failed-login message, so the **hydra** distinguishes the correct password from the incorrect one.

```
(rodis@kali)~[~/Desktop/passwords]
$ hydra -l admin -P ./password.txt 192.168.2.1 http-get-form "/log/in:un=admin&pw=^PASS^&rd=%2Fuir%2Fdw
home.htm&rd2=%2Fuir%2Floginpage.htm&Nrd=1&Nlmb= HTTP/1.1:invalid username or password" -t 2 -vV -e s
```

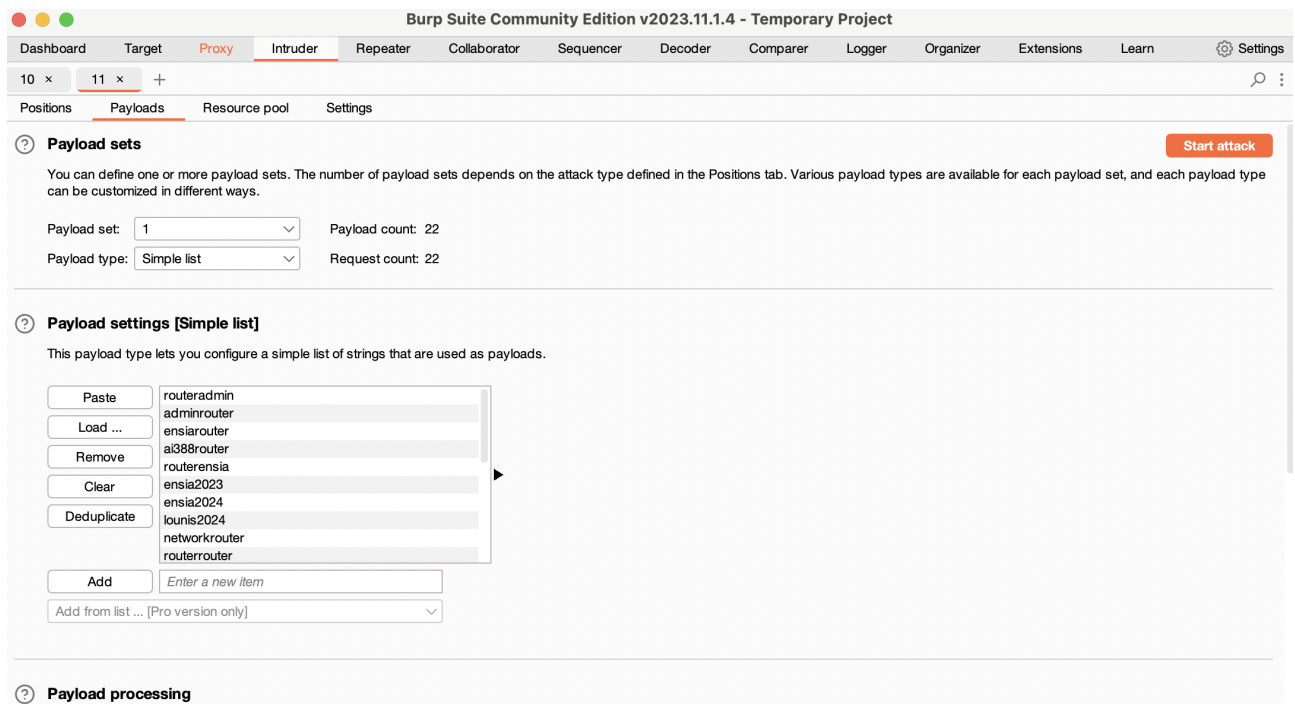
- If the server becomes non-responsive, you may ask your lab instructor to reboot it. The web-server may have crashed, although the router may still respond to your ICMP pings. You should make sure to launch the attack with parameter **-t 2**. If you bomb the router with **-t 50**, it may crash, and the cracking fails.

Task 2 (Cracking Credentials with Burp Suite Community Edition)

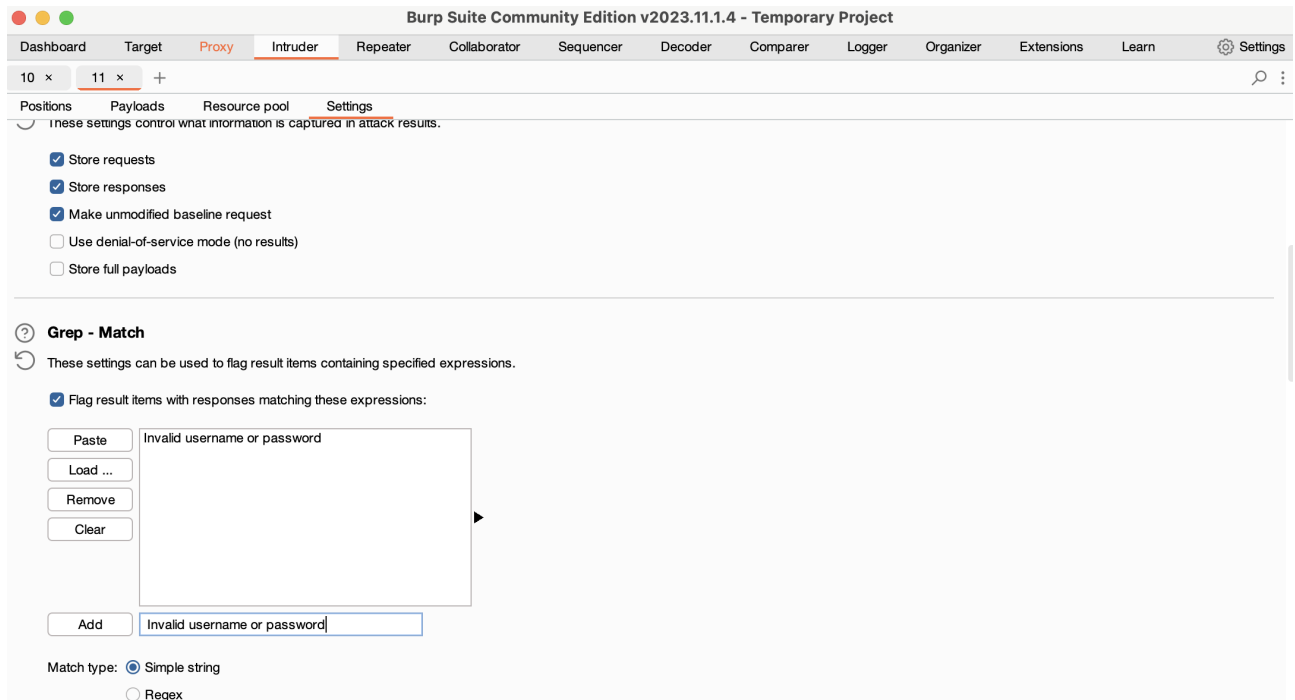
In this part, you will use the **Burp Suite Community Edition** to crack the credentials of the web-server running on the router **AI388 Lab**.

- First of all, you need to run **burp** and turn on the proxy interceptor. Launch the **burp**'s browser and input the IP address of the router (**AI388 Lab**).
- The tool will intercept the request. You should click on “Forward” until you receive the login page.
- Type in incorrect credentials (password) and click on “Login”. You should see your HTTP POST request along with the credentials.
- Click on “Action”, then “Send to Intruder”. Click on “Intruder” tab.
- You should see your HTTP request under the “Payload positions” section.
- Select the value of the password parameter and click on “Add §” (on the right).

- Then go to “Payload” tab and under “Payload setting”, click on Load and choose your password file (**passwords.txt**), and click open.



- Then go to “Setting” tab and under “Grep - Match”, click on Clear, then write the failed login message on the input field and click Add.



- Now, scroll up, and if everything is good, your should be able to start the attack by clicking on “Start attack”.

15. Intruder attack of http://192.168.2.1 - Temporary attack - Not saved to project file

Results

Positions

Payloads

Resource pool

Settings

▼

Filter: Showing all items

Request ^	Payload	Status code	Error	Timeout	Length	Invalid ...	Comment	
0		303	☐	☐	109			
1	routeradmin	303	☐	☐	109			
2	adminrouter	303	☐	☐	109			
3	ensiarouter	303	☐	☐	109			
4	ai388router	303	☐	☐	109			
5	routerensia	303	☐	☐	109			
6	ensia2023	303	☐	☐	109			
7	ensia2024	303	☐	☐	109			
8	lounis2024	303	☐	☐	109			
9	networkrouter	303	☐	☐	109			
10	routerrouter	303	☐	☐	109			
11	adminROUTER	303	☐	☐	109			
12	ROUTERrouter	303	☐	☐	109			
13	lounis23	303	☐	☐	109			
14	lounis24	303	☐	☐	148			
15	lounisrouter	303	☐	☐	109			
16	routerAlensia	303	☐	☐	109			
17	ensia2024router	303	☐	☐	109			
18	tcpiprouterensia	303	☐	☐	109			
19	ensia.dz@router	303	☐	☐	109			
20	router@ensia.dz	303	☐	☐	109			
21	ai388.router@ensia	303	☐	☐	109			
22		303	☐	☐	109			

- Analyze the result of the attack by checking the different HTTP requests and their corresponding HTTP response and try to figure out the password. The tool many not spot the failed-login message in the HTTP responses, but there could be another indicator that differentiate the different responses.
- What would you recommend as countermeasures to harden the cracking process for an attacker? Discuss.

15. Intruder attack of http://192.168.2.1 - Temporary attack - Not saved to project file

Results

Positions

Payloads

Resource pool

Settings

Filter: Showing all items

Request ^	Payload	Status code	Error	Timeout	Length	Invalid ...	Comment
0		303	☐	☐	109		
1	routeradmin	303	☐	☐	109		
2	adminrouter	303	☐	☐	109		
3	ensiarouter	303	☐	☐	109		
4	ai388router	303	☐	☐	109		
5	routerensia	303	☐	☐	109		
6	ensia2023	303	☐	☐	109		
7	ensia2024	303	☐	☐	109		
8	lounis2024	303	☐	☐	109		
9	networkrouter	303	☐	☐	109		
10	routerrouter	303	☐	☐	109		
11	adminROUTER	303	☐	☐	109		
12	ROUTERrouter	303	☐	☐	109		
13	lounis23	303	☐	☐	109		
14	lounis24	303	☐	☐	148		
15	lounisrouter	303	☐	☐	109		
16	routerAlensia	303	☐	☐	109		
17	ensia2024router	303	☐	☐	109		
18	tcpiprouterensia	303	☐	☐	109		

Request

Response

Pretty

Raw

Hex

Render

1 HTTP/1.1 303

2 Content-Type: text/html

3 Location: /uir/loginpage.htm?Nrc=1&Nrd=1&Nlmb=

4 Connection: close

5

Search

0 highlights

Finished

Task 3 (Cracking with John the Ripper) — Offline Cracking

John the Ripper (JtR) is an offline password cracking tool that supports a diversity of password formats. The password may be of encrypted, hashed (e.g., message authentication codes (MACs) and hash-based MACs (HMACs)).

It supports three different cracking modes: (1) Single crack mode, which uses information about the user, such as username, from the supplied password values, where present, to attempt to build a short, highly focused dictionary of targeted passwords to crack. (2) Wordlist mode, operates via a wordlist, either user-supplied or a built-in one, and tries all the values in the wordlist, and (3) Incremental mode, attempts to iterate through all possible character combinations according to adjustable length and complexity criteria.

1. **Cracking GNU/Linux password.** In this section, you will use John to crack the password of a Linux user (i.e., an offline password cracking).

- First of all, start by creating a Linux user (e.g., **target**) and give them a password, let us say, **ensia1999** (recall, **\$useradd target** allows you to create a user named **target** and **\$passwd target** allows you to set up a password for the user **target**).
- On GNU/Linux systems, users' information are stored in **/etc/passwd** and their respective hashed passwords in **/etc/shadow**. Modern Linux systems use the **bcrypt** hash function to hash the users' passwords.
- To use John, you would copy the relevant line from the **/etc/shadow** file and save it into a new text file, e.g., **passwordHash.txt**. The line looks similar to the following:

```
target:$y$j9T$3IKH...:1001:1001::/home/target:/bin/sh
```

- As you can see here, the **target** is the username, **\$y\$** indicates that **bcrypt** was used to hash the password, and the **j9T\$...** is the password hash, and the remaining numbers are password and user account settings.
- Now, that the hash is in a safe file (**passwordHash.txt**), let us instruct **John** to crack the password given the hash. **John** will determine which hash function to use to crack the password for a given user. Run the following **John** command (make sure that you are located in a folder where your password list is located):

```
$sudo john --format=crypt --wordlist=./wplist.txt  
passwordHash.txt
```

- **John** will use multi-threading to crack the password. After few seconds, **John** will crack and display the username and password.

2. **Cracking Zip/Pdf password.** In this section, you will use **John** to crack the password a protected file (i.e., offline password cracking).

- Assuming you have captured a password-protected zip folder, **SecretBox.zip** that contains a password-protected pdf file, which in turn contains secret information. You will use **John** to crack the passwords.
- You should start by cracking the password that protect the zipped folder. To that end, you first need to extract the hash of the password, that is part of the folder content. Use the following commands:

```
$sudo zip2john SecretFile.zip > zipHash
```

- Once you've got the hash (you can check its content `$cat zipHash`), you can instruct **John** to use your password list to try cracking the password:

```
$sudo john --wordlist=./wplist.txt pdfHash
```

- **John** will identify the hash function and use the appropriate one to crack the password. After few seconds, you should be able to see the password and you should be able to extract the content of the zipped folder.
- Use **John** again to crack the password of the PDF file. The instructions are the same as above, but instead of collecting the hash using **zip2john**, use **pdf2john**. What are the two passwords, and what is the content of the file?

Note. If **John** cracked your passwords and you wanted to crack them again, you may first have to delete the following two files: **john.rec** and **john.pot**, both located in **/root/.john/**. Also, on Kali/Linux, you can access various password lists: check their path by typing `$wordlist -h`.

Another tool that you can explore at home to perform the same cracking attacks as above, is the **Hashcat** password cracking tool (10500 is the code of the hash function that was used (e.g., `$pdf$2*3*128*`). You can check the complete list [here](#)):

```
$sudo hashcat -m 10500 pdfHash wplist.txt
```